

Εργασία Hardware I

Γκέντζη Αλεξάνδρα

Περιεχόμενα

Άσκηση 1.....	3
Άσκηση 2.....	4
Άσκηση 3.....	5
Άσκηση 4.....	5

Τα αρχεία είναι γραμμένα σε system Verilog γιατί αντιμετωπίσα κάποια προβλήματα με τα αρχεία σε απλή Verilog.

Άσκηση 1

Στα πλαίσια της άσκησης 1 ζητήθηκε η υλοποίηση μίας αριθμητικής/λογικής μονάδας (ALU) προσημασμένων αριθμών, εύρους 32-bit, η οποία να μπορεί να εκτελεί τις παρακάτω πράξεις μεταξύ δύο τελεστών (op1, op2). Η κωδικοποίηση των πράξεων έχει ως εξής στο δυαδικό σύστημα:

Πράξη	Κωδικοποίηση
Λογική AND	1000
Λογική OR	1001
Λογική NOR	1010
Λογική NAND	1011
Λογική XOR	1100
Προσημασμένη Πρόσθεση	0100
Προσημασμένη Αφαίρεση	0101
Προσημασμένος Πολλαπλασιασμός	0110
Λογική ολίσθηση δεξιά	0000
Λογική ολίσθηση αριστερά	0001
Αριθμητική ολίσθηση δεξιά	0010
Αριθμητική ολίσθηση αριστερά	0011

Αρχικά οι διαφορετικές λειτουργίες ορίστηκαν ως παράμετροι στο module alu, ενώ το συνδυαστικό κύκλωμα υλοποιήθηκε με την χρήση ενός always block, μέσα στο οποίο η εντολή case λειτουργεί ως πολυπλέκτης για την επιλογή της λειτουργίας που θα εκτελεστεί.

Δεδομένου ότι εργαζόμαστε με προσημασμένους αριθμούς, ελέγχουμε την εμφάνιση του overflow ως εξής:

- **Πρόσθεση:** Υπερχείλιση στην πρόσθεση μπορεί να προκύψει μόνο αν οι προσθετέοι έχουν ίδιο πρόσημο. Συνεπώς, ο κώδικας ελέγχει αν τα MSB των προσθετέων είναι ίδια και αν το MSB του αποτελέσματος διαφέρει.

```
ovf = (op1[31] == op2[31]) && (result[31] != op1[31]);
```

- **Αφαίρεση:** Αντίστοιχα στην αφαίρεση έχουμε υπερχείλιση μόνο όταν οι δύο τελεστές έχουν διαφορετικά πρόσημα. Ο κώδικας ελέγχει αν τα MSB των τελεστών είναι διαφορετικά και αν το πρόσημο του αποτελέσματος είναι διάφορο από το πρόσημο του μειωτέου (op1).

```
ovf = (op1[31] != op2[31]) && (result[31] != op1[31]);
```

- **Πολλαπλασιασμός:** Παρόλο που το γινόμενο δύο 32-bit αριθμών παράγει αποτέλεσμα 64-bit στην περίπτωση μας κρατούνται μόνο τα 32 LSB για αυτό και πρέπει να εξασφαλίσουμε ότι στα 32 MSB δεν υπάρχει χρήσιμη πληροφορία παρά μόνο επέκταση πρόσημου.

```
ovf = (mult_full[63:32] != {32{mult_full[31]}});
```

Άσκηση 2

Για την άσκηση αυτή σχεδιάστηκε ένα κύκλωμα αριθμομηχανής το οποίο χρησιμοποιεί την ALU από την Άσκηση 1.

Όσον αφορά το αποτέλεσμα της αριθμομηχανής (accumulator) χρησιμοποιήθηκε ένα always block το οποίο εξασφαλίζει ότι ο μηδενισμός και η ανανέωση της τιμής του γίνεται σύγχρονα με τα σήματα btnac και btnc αντίστοιχα.

```
always @(posedge clk) begin
    if(btnac)
        accumulator <= 16'b0;
    else if(btnc)
        accumulator <= alu_result[15:0];
end
```

Η επιλογή της πράξης που θα εκτελεστεί κάθε φορά εξαρτάται από τις τιμές των πλήκτρων btnl, btnr, btnd σύμφωνα με τα συνδυαστικά κυκλώματα που δόθηκαν στην εκφώνηση. Επομένως για την απλοποίηση του κώδικα δημιουργήθηκε το αρχείο calc_enc.sv στο οποίο υπολογίζονται τα 4 bit του τελεστή alu_op, υπεύθυνο για την επιλογή πράξης.

Διάγραμμα χρονισμού



Όπως παρατηρείται στο διάγραμμα χρονισμού:

- Για τον έλεγχο της λειτουργίας του RESET ενεργοποιείται το σήμα του btnac με αποτέλεσμα τον μηδενισμό του led στην επόμενη θετική ακμή του ρολογιού.
- Οι αναμενόμενες τιμές (expected_values) εμφανίζονται άμεσα εφόσον δεν εξαρτώνται από το ρολόι.
- Για την εγγραφή του νέου αποτελέσματος στον accumulator ενεργοποιείται το σήμα btnc και η ανανέωση της τιμής εμφανίζεται στην επόμενη θετική ακμή του ρολογιού. Ο accumulator αποθηκεύει τα 16 LSB του alu_result γεγονός που εξηγεί τη διαφορά στις τιμές μεταξύ τους.

Άσκηση 3

Σε αυτήν την άσκηση δημιουργήθηκε ένα αρχείο 16 καταχωρητών το οποίο αποθηκεύει τις τιμές των καταχωρητών που χρησιμοποιούνται από το σύστημα.

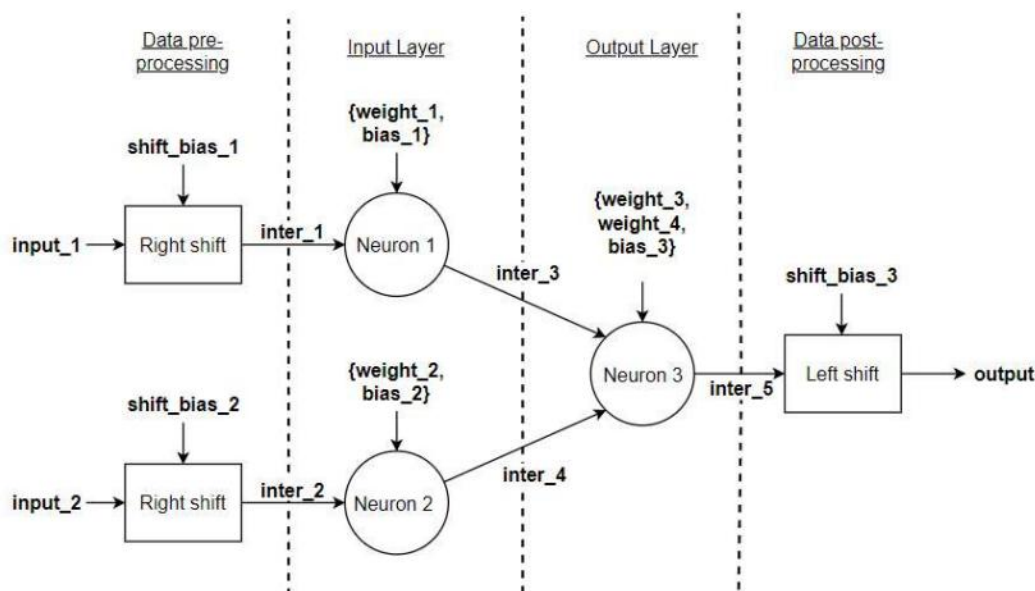
Μέσω του always block στο αρχείο regfile.sn εξασφαλίζεται η ασύγχρονη λειτουργία του RESET, ώστε κατά την ενεργοποίηση (resetn = 0, active low reset) του να μηδενίζονται άμεσα όλοι οι καταχωρητές. Όπως φαίνεται, η διαδικασία εγγραφής στο αρχείο είναι συγχρονισμένη με το ρολόι και επιτρέπει ταυτόχρονη ενημέρωση μόνο δύο θέσεων μνήμης σε κάθε κύκλο ρολογιού, μέσω των writeReg1 και writeReg2. Σε αντίθεση, η διαδικασία ανάγνωσης γίνεται ασύγχρονα μέσω της εντολής assign.

Επιπλέον για την αντιμετώπιση του προβλήματος της ταυτόχρονης εγγραφής και ανάγνωσης στην ίδια διεύθυνση προστέθηκε κώδικας ο οποίος ελέγχει αν είναι ενεργοποιημένο το σήμα write, αν οι διευθύνσεις είναι ίδιες και δίνει προτεραιότητα στην εγγραφή.

```
assign readData1 = (write && writeReg1 == readReg1) ? writeData1 :  
                  (write && writeReg2 == readReg1) ? writeData2 :  
                  regs[readReg1];
```

Άσκηση 4

Χρησιμοποιώντας όσα υλοποιήθηκαν στις παραπάνω ασκήσεις σχεδιάστηκε το νευρωνικό σύστημα του παρακάτω σχήματος:



Για την υλοποίηση των αριθμητικών μεταποιήσεων στα Data pre-processing και Data post-processing χρησιμοποιήθηκαν δύο ALU όπως λέει ρητά στην εκφώνηση.

Επίσης κάθε νευρώνας πραγματοποιεί δύο πράξεις διαδοχικά, έναν πολλαπλασιασμό με το βάρος και μία πρόσθεση με την πόλωση. Με σκοπό την απλοποίηση του κώδικα

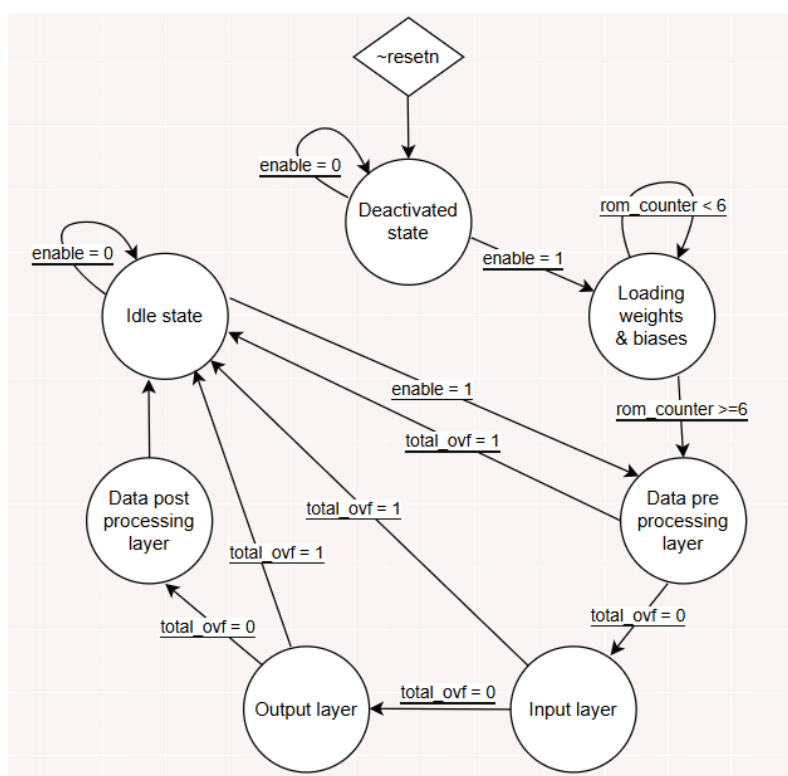
δημιουργήθηκε το ξεχωριστό αρχείο mac_unit.sv που υλοποιεί τις δύο αυτές πράξεις με την χρήση δύο ALU την μία μετά την άλλη.

Στην συνέχεια τα στάδια του νευρωνικού δικτύου υλοποιήθηκαν με την χρήση ενός FSM το οποίο έχει 7 στάδια, συγκεκριμένα:

- **Deactivated State (000):** Το νευρωνικό σύστημα δεν εκτελεί πράξεις. Όσο το resetn βρίσκεται σε χαμηλή κατάσταση παραμένει στο deactivated state, ενώ όταν ενεργοποιηθεί το σήμα enable και το resetn μεταβεί στην υψηλή τάση μεταβαίνει στην κατάσταση Loading weights & biases.
- **Loading weights & biases (001):** Φορτώνει τα βάρη και τις πολώσεις από την ROM στο register file.
- **Data pre-processing layer (010):** Επεξεργάζεται τις εισόδους με την χρήση των alu_1 και alu_2.
- **Input layer (011):** Οι είσοδοι περνάνε από τους δύο παράλληλους νευρώνες, σε περίπτωση υπερχείλισης μεταβαίνει στο Idle state.
- **Output layer (100):** Περνάει τα αποτελέσματα του input layer από έναν νευρώνα, σε περίπτωση υπερχείλισης μεταβαίνει στο Idle state.
- **Data post-processing layer (101):** Επεξεργάζεται το αποτέλεσμα του output layer με την χρήση της alu_1 και σε κάθε περίπτωση μεταβαίνει στο Idle state.
- **Idle state (110):** Περιμένει τις επόμενες εισόδους και μόλις ενεργοποιηθεί το σήμα enable μεταβαίνει στην κατάσταση Data pre-processing layer.

Όταν ενεργοποιηθεί το σήμα RESET (resetn = 0, active low reset) μεταβαίνει στο Deactivated state, σε οποιοδήποτε κατάσταση και αν βρίσκεται.

Διάγραμμα Μεταβάσεων Κατάστασης



Συγκεκριμένα υλοποιήθηκε ένα 3-blocks FSM τύπου Moore εφόσον οι έξοδοι εξαρτώνται μόνο από την τρέχουσα κατάσταση.

Για την αποθήκευση των δεδομένων επιλέχθηκε η χρήση των ενδιάμεσων registers και όχι η αποθήκευση στο register file για μεγαλύτερη ταχύτητα και μείωση της πολυπλοκότητας. Συγκεκριμένα χρησιμοποιήθηκαν οι τελεστές `inter_1`, `inter_2` για την αποθήκευση των αποτελεσμάτων των `alu_1` και `alu_2` αντίστοιχα στο στάδιο Data pre-processing layer, οι `inter_3`, `inter_4` για την αποθήκευση των αποτελεσμάτων των `mac_1` και `mac_2` αντίστοιχα στο στάδιο Input layer και ο `inter_5` για την αποθήκευση του αποτελέσματος του `mac_2` στο Output layer. Επομένως, το register file χρησιμοποιήθηκε μόνο για την αποθήκευση των βαρών και των πολώσεων.

Το **STATE_MEMORY block** είναι το μοναδικό block που συνδέεται με το ρολόι και το σήμα `resetn`. Σε αυτό πραγματοποιείται η ενημέρωση της τρέχουσας κατάστασης και η αποθήκευση των ενδιάμεσων τιμών αλλά και των τελεστών υπερχείλισης (`total_ovf`) και μηδενισμού (`total_zero`). Χρησιμοποιούνται αποκλειστικά non-blocking εντολές ώστε όλες οι αλλαγές να γίνονται ταυτόχρονα στην επόμενη θετική ακμή του ρολογιού. Έτσι, εξασφαλίζεται ότι τα δεδομένα μένουν αποθηκευμένα για να χρησιμοποιηθούν στα επόμενα στάδια του κυκλώματος.

Στην συνέχεια, το **NEXT_STATE_LOGIC block** είναι υπεύθυνο για την άμεση επιλογή της επόμενης κατάστασης με βάση την τωρινή κατάσταση και τα σήματα `total_ovf`, `enable`, `rom_counter`, για αυτό και υλοποιήθηκε με την χρήση της εντολής `case`, ώστε να ελέγχει σε ποια κατάσταση βρίσκεται το σύστημα.

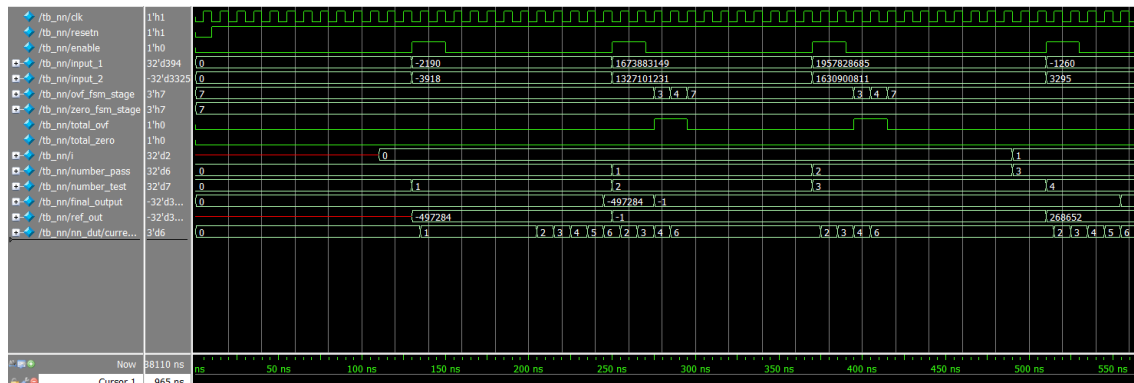
Τέλος το **OUTPUT_LOGIC block** αποτελείται από blocking εντολές ώστε οι έξοδοι και τα σήματα να υπολογίζονται άμεσα σύμφωνα με την τωρινή κατάσταση. Συγκεκριμένα στην κατάσταση Loading weights & biases υλοποιείται η ανάγνωση δεδομένων από την ROM και η εγγραφή τους στις αντίστοιχες θέσεις του register file. Επειδή σε κάθε κύκλο ρολογιού μπορούν να εγγραφούν ταυτόχρονα μόνο δύο τιμές στο register file έγινε χρήση της μεταβλητής `rom_counter`. Όπως φαίνεται από τον κώδικα, η εγγραφή στο register file ξεκινάει για `rom_counter > 0`. Αυτό συμβαίνει γιατί η ROM είναι σύγχρονη, οπότε απαιτείται ένας κύκλος ρολογιού από την στιγμή που δίνεται η διεύθυνση μέχρι να εμφανιστούν τα δεδομένα στην έξοδο. Με σκοπό την αντιμετώπιση αυτής της καθυστέρησης χωρίς να μειωθεί η ταχύτητα φόρτωσης των δεδομένων, σε κάθε κύκλο ρολογιού ζητούνται από την ROM τα επόμενα δεδομένα και ταυτόχρονα γράφονται στο register file τα δεδομένα που μόλις εμφανίστηκαν. Εφόσον τα δεδομένα εγγραφής είναι 10 και ο χρόνος καθυστέρησης ένας κύκλος, χρειάζονται συνολικά 6 κύκλοι ρολογιού για την ολοκλήρωση της διαδικασίας (`rom_counter < 6`). Στις υπόλοιπες καταστάσεις αντλούνται τα κατάλληλα δεδομένα από το register file και πραγματοποιούνται οι απαραίτητες πράξεις κάθε σταδίου.

Για την επαλήθευση της σωστής λειτουργία του νευρωνικού δικτύου υλοποιήθηκε το αρχείο `tb_nn.sv` στο οποίο συγκρίνονται τα αποτελέσματα του `nn.sv` με το `nn_model.sv`. Αρχικά λήφθηκαν υπόψιν οι καθυστερήσεις του κυκλώματος ώστε να

υπάρχει αρκετός χρόνος για την ολοκλήρωση κάθε σύγκρισης (#100 = 10 clock cycles) ενώ για λόγους απασφαλμάτωσης (debugging) προστέθηκαν μηνύματα επιβεβαίωσης τόσο για τα λανθασμένα όσο και για τα σωστά αποτελέσματα.

Στην συνέχεια πραγματοποιήθηκαν 300 τεστ, 100 για κάθε ένα από τα τρία διαφορετικά διαστήματα επιλογής ζεύγους εισόδων που δόθηκαν στην εκφώνηση. Ο τελικός κώδικας κατάφερε να επιτύχει 300/300 επιτυχείς τεστ.

Διάγραμμα Χρονισμού



Όπως φαίνεται και στο διάγραμμα χρονισμού (4 τεστ περίπου από τα 300), πριν από κάθε σύγκριση ενεργοποιείται το σήμα enable για την εισαγωγή των νέων εισόδων και απενεργοποιείται μετά από 2 κύκλους ρολογιού. Η διαδικασία αυτή έγινε με σκοπό την εκτέλεση των υπολογισμών ακριβώς μία φορά για κάθε ζευγάρι εισόδων.